CAMPUSCARE

AN ONLINE CAMPUS COMPLAINT MANAGEMENT SYSTEM

PROJECT REPORT

Submitted by:
Soumya Swastik Sahoo
Registration Number: 250301120048

Department: _____
Institution: _____
Project Guide: _____
Academic Year: _____

DECLARATION

I, Soumya Swastik Sahoo, bearing Registration Number 250301120048, declare that the project titled “CampusCare: An Online Campus Complaint Management System” is my academic project work. The project was developed for the purpose of recording, managing, and tracking campus-related complaints through a RESTful backend application. The work presented in this report is prepared for academic submission under the guidance of the concerned faculty member.

ACKNOWLEDGEMENT

I express my sincere gratitude to my project guide, faculty members, and institution for their guidance and support during the development of CampusCare. I also thank everyone who contributed suggestions and feedback regarding the organization of campus complaints and the design of the application. Finally, I am grateful to my family and friends for their encouragement throughout the project.

ABSTRACT

CampusCare is a web-based campus complaint management system designed to provide a structured method for submitting, storing, retrieving, updating, and tracking complaints. In a traditional campus environment, complaints about food, hostel rooms, cleanliness, water, electricity, maintenance, and other facilities may be communicated verbally or through informal channels. These methods can make it difficult to maintain records, monitor progress, and identify recurring problems.

The implemented system provides a RESTful application programming interface through which complaints can be created and managed. Each complaint contains information such as type, category, description, status, requested resolution time, tracking identifier, creation time, and optional photo information. The application uses Spring Boot, Spring Web, Spring Data JPA, Jakarta Validation,

and MySQL. A layered structure separates the controller, service, repository, model, and exception-handling responsibilities.

The system supports complaint creation, retrieval of all complaints, retrieval by database identifier, retrieval by tracking identifier, filtering by category, updating, and deletion. Validation rules help ensure that required fields are present and that the description and requested resolution time meet minimum requirements. The project provides a foundation that can be extended with user authentication, role-based access, dashboards, notifications, file storage, and advanced reporting.

Keywords: CampusCare, complaint management, REST API, Spring Boot, MySQL, Spring Data JPA, validation, complaint tracking.

TABLE OF CONTENTS

1. Introduction
2. Background and Problem Statement
3. Objectives
4. Scope of the Project
5. Feasibility Study
6. Requirement Analysis
7. System Design
8. Technology Used
9. Implementation
10. API Specification
11. Testing and Results
12. Advantages and Limitations
13. Future Enhancements
14. Conclusion
15. References

1. INTRODUCTION

Educational institutions manage a variety of services and facilities, including hostels, food services, electrical systems, water supply, cleanliness, and general maintenance. Students and staff need a reliable way to report issues and follow their progress. CampusCare addresses this need by providing a centralized complaint-management backend.

The project is implemented as a Spring Boot application. It exposes HTTP endpoints under the /api/complaints path. The design follows a layered architecture. The controller receives HTTP requests, the service layer applies business operations, the repository communicates with the database, and the entity model represents complaint data.

The main purpose of CampusCare is not only to store complaints, but also to make them identifiable and trackable. A unique tracking identifier is generated when a complaint is first saved, and a creation timestamp is recorded

automatically.

2. BACKGROUND AND PROBLEM STATEMENT

2.1 Background

Many campus complaints are handled through conversations, paper forms, phone calls, or unrelated messaging groups. Although these channels may be useful for urgent communication, they do not always provide a dependable central record. As a result, complaints may be forgotten, duplicated, delayed, or difficult to measure.

2.2 Problem Statement

The problem is the absence of a simple, centralized, and trackable mechanism for recording and managing campus complaints. The proposed system should allow authorized application clients to submit complaints, preserve their details, retrieve them when required, update their status or information, and locate a complaint through a tracking identifier.

2.3 Proposed Solution

CampusCare provides a REST API connected to a MySQL database. The API accepts validated complaint data, persists it through JPA, and exposes operations for retrieval, filtering, updating, and deletion. The design can be connected to a web or mobile frontend in the future.

3. OBJECTIVES

The objectives of CampusCare are:

1. To provide a centralized system for storing campus complaints.
2. To allow a client application to create a complaint using a REST endpoint.
3. To generate a unique tracking identifier for each new complaint.
4. To record the date and time at which a complaint is created.
5. To support retrieval of all complaints and individual complaints.
6. To allow complaints to be searched by tracking identifier.
7. To allow complaints to be filtered by category.
8. To support updating and deleting complaint records.
9. To validate important input fields before database persistence.
10. To provide a maintainable backend that can support future user interfaces.

4. SCOPE OF THE PROJECT

4.1 Included Scope

The current implementation includes:

• Complaint creation through a POST request.
• Retrieval of all complaints.
• Retrieval of a complaint by numeric ID.
• Retrieval of a complaint by tracking ID.
• Filtering of complaints by category.
• Updating of complaint information.
• Deleting of a complaint.
• Complaint validation using Jakarta Validation.
• Automatic tracking ID and creation-time generation.
• Optional complaint photo metadata and stored photo data.
• MySQL persistence through Spring Data JPA.
• Validation error responses through a global exception handler.

4.2 Excluded or Not Yet Implemented

The current source code does not show a complete user-interface module, authentication system, role-based authorization, email/SMS notification service, administrator dashboard, or analytics module. These are recommended future enhancements and should not be described as completed features unless they are implemented separately.

5. FEASIBILITY STUDY

5.1 Technical Feasibility

The application uses established Java and Spring technologies. Spring Boot reduces configuration effort, Spring Web supports REST endpoints, Spring Data JPA simplifies database access, and MySQL provides relational data storage. Therefore, the system is technically feasible for a small or medium campus deployment.

5.2 Operational Feasibility

The complaint workflow is simple: a client submits information, the system stores it, and the client can retrieve or update it. This makes the solution easy to integrate with a future web or mobile interface.

5.3 Economic Feasibility

The project uses open-source technologies and does not require specialized hardware. The primary requirements are a Java development environment, Maven, and a MySQL server. This makes the solution economically feasible for an academic prototype.

5.4 Schedule Feasibility

The project can be developed incrementally. The database entity and repository can be created first, followed by service operations, REST endpoints, validation, testing, and frontend integration.

6. REQUIREMENT ANALYSIS

6.1 Functional Requirements

FR1: The system shall accept a new complaint.
FR2: The system shall validate required complaint fields.
FR3: The system shall generate a tracking ID for a new complaint.
FR4: The system shall record the complaint creation time.
FR5: The system shall return all stored complaints.
FR6: The system shall return one complaint by its ID.
FR7: The system shall return one complaint by its tracking ID.
FR8: The system shall return complaints belonging to a category.
FR9: The system shall update an existing complaint.
FR10: The system shall delete an existing complaint.
FR11: The system shall return a client error for invalid complaint input.

6.2 Non-Functional Requirements

- Maintainability: Responsibilities are separated into layers.
- Reliability: Data is persisted in a relational database.
- Usability: The REST API can be consumed by different client applications.
- Extensibility: Additional authentication, notifications, and dashboards can be added without replacing the core complaint model.
- Data integrity: Validation constraints reduce incomplete or invalid records.
- Performance: Spring Data repository methods provide standard database operations suitable for the prototype scale.

6.3 Complaint Data Requirements

Each complaint may contain:

- id: database-generated numeric identifier.
- trackingId: UUID-based tracking identifier.
- createdAt: date and time of creation.
- requestedResolutionHours: minimum requested resolution period.
- type: complaint type.
- category: complaint category.
- description: detailed complaint text.
- status: current complaint status.
- photoData: optional photo content.
- photoFileName: optional original file name.
- photoMimeType: optional MIME type.

Supported category values represented by the project include FOOD, HOSTEL_ROOM, CLEANLINESS, WATER, ELECTRICITY, MAINTENANCE, and OTHER.

7. SYSTEM DESIGN

7.1 Architecture

CampusCare follows a layered architecture:

Client Layer
Sends HTTP requests and receives JSON responses.

Controller Layer
Maps HTTP requests to complaint operations.

Service Layer
Coordinates complaint creation, retrieval, update, and deletion.

Repository Layer
Uses Spring Data JPA to access the complaint table.

Database Layer
Stores complaint records in MySQL.

7.2 Data Flow for Creating a Complaint

1. A client sends a POST request to /api/complaints.
2. Spring converts the request body into a complaint object.
3. Jakarta Validation checks required fields and minimum values.
4. The controller checks that requested resolution hours are present.
5. The service passes the complaint to the repository.
6. Before persistence, the entity generates a tracking ID and creation time.
7. JPA saves the record in MySQL.
8. The saved complaint is returned to the client.

7.3 Entity Relationship Description

The current application contains one primary persistent entity: complaint. Each complaint is represented by one row in the complaint table. The ID is generated by the database. The tracking ID is generated by the application. The photo data is mapped as a large object and the other fields are stored as regular columns.

7.4 Security Considerations

The current prototype permits cross-origin requests from all origins to support early client integration. A production deployment should restrict allowed origins and add authentication and authorization. Database credentials must be kept outside source control and supplied through environment variables or a secure configuration service.

8. TECHNOLOGY USED

8.1 Programming Language

Java is used for application development. The project declares Java 25 as its

configured language version.

8.2 Framework

Spring Boot is used to bootstrap the application and provide application configuration, dependency injection, web support, and production packaging.

8.3 REST and Web

Spring Web provides REST controller annotations such as @RestController, @RequestMapping, @GetMapping, @PostMapping, @PutMapping, and @DeleteMapping.

8.4 Persistence

Spring Data JPA and Hibernate provide object-relational mapping and repository operations. MySQL is used as the runtime database.

8.5 Validation

Jakarta Validation annotations such as @NotBlank, @Size, and @Min validate incoming complaint data. A global exception handler returns validation errors with HTTP 400 Bad Request.

8.6 Build Tool

Maven manages project dependencies, compilation, testing, and Spring Boot packaging.

9. IMPLEMENTATION

9.1 Application Entry Point

CampuscareApplication starts the Spring Boot application. Component scanning discovers the controller, service, repository, entity, and exception-handler classes.

9.2 Complaint Entity

The complaint entity represents the core business object. Its fields describe the issue, classification, status, time requirement, tracking information, and optional image information. The @PrePersist method assigns a UUID tracking ID and current timestamp when values have not already been set.

9.3 Controller

ComplaintController exposes the /api/complaints resource. It delegates business operations to ComplaintService rather than directly accessing the database. This keeps HTTP concerns separate from persistence concerns.

9.4 Service

ComplaintService contains operations for saving, listing, updating, deleting, and finding complaints. During an update, existing values are loaded first. Optional photo fields and requested resolution hours are replaced only when corresponding values are supplied.

9.5 Repository

ComplaintRepository extends JpaRepository<complaint, Long>. It inherits common CRUD operations and declares finder methods for tracking ID, status, and category.

9.6 Exception Handling

GlobalExceptionHandler handles MethodArgumentNotValidException and returns a map containing field names and validation messages. This gives API clients structured information about invalid input.

10. API SPECIFICATION

Base path: /api/complaints

10.1 Create Complaint

Method: POST
Path: /api/complaints
Purpose: Create a complaint.
Request body: JSON complaint object.
Success response: Saved complaint object.
Validation response: HTTP 400 Bad Request.

Example request:

```
{
  "type": "Facility",
  "category": "WATER",
  "description": "There is no water supply in the hostel block.",
  "status": "OPEN",
  "requestedResolutionHours": 24
}
```

10.2 Get All Complaints

Method: GET
Path: /api/complaints
Purpose: Return all complaints.

10.3 Get Complaint by ID

Method: GET
Path: /api/complaints/{id}

Purpose: Return one complaint by database ID.

10.4 Track Complaint

Method: GET
Path: /api/complaints/track/{trackingId}
Purpose: Return one complaint using its generated tracking ID.

10.5 Filter by Category

Method: GET
Path: /api/complaints/category/{category}
Purpose: Return complaints matching a category.

10.6 Update Complaint

Method: PUT
Path: /api/complaints/{id}
Purpose: Update an existing complaint.

10.7 Delete Complaint

Method: DELETE
Path: /api/complaints/{id}
Purpose: Delete an existing complaint.
Success response: “Complaint deleted successfully”.

11. TESTING AND RESULTS

11.1 Testing Approach

Testing verifies that the application context can start and that the project can be built using its Maven configuration. API testing should additionally verify successful requests, invalid requests, missing records, update operations, category filtering, and deletion.

11.2 Implemented Automated Test

The project contains a Spring Boot context-loading test. This test checks whether the application can initialize its Spring context successfully.

11.3 Recommended API Test Cases

TC01: Submit a valid complaint. Expected result: complaint is saved and a tracking ID is returned.

TC02: Submit a complaint without a type. Expected result: HTTP 400 with a type validation message.

TC03: Submit a complaint with a description shorter than ten characters.

Expected result: HTTP 400 with a description validation message.

TC04: Submit a complaint with requestedResolutionHours below one. Expected result: HTTP 400 with a minimum-value validation message.

TC05: Retrieve all complaints. Expected result: HTTP 200 and a list.

TC06: Retrieve a valid complaint ID. Expected result: HTTP 200 and one object.

TC07: Retrieve an unknown complaint ID. Expected result: complaint-not-found error behavior.

TC08: Track a complaint using its tracking ID. Expected result: HTTP 200 and the matching complaint.

TC09: Filter by category. Expected result: complaints from the selected category.

TC10: Update a complaint. Expected result: modified complaint is persisted.

TC11: Delete a complaint. Expected result: the record is removed and a success message is returned.

11.4 Expected Result

The application is expected to start when the configured MySQL database is available and the database credentials are supplied correctly. The API operations are implemented through the controller, service, and repository layers.

12. ADVANTAGES AND LIMITATIONS

12.1 Advantages

- Centralized complaint storage.
- Unique tracking IDs improve complaint identification.
- Category filtering supports organized issue management.
- Validation reduces incomplete submissions.
- REST architecture allows web and mobile integration.
- Layered design improves maintainability.
- MySQL provides persistent relational storage.

12.2 Limitations

- The current source code does not include authentication or user roles.
- The API currently allows all cross-origin requests.
- The project has limited automated endpoint tests.
- Runtime database configuration depends on a local MySQL server.
- Photo content is stored directly in the entity as large text data, which may not be ideal for large-scale production use.

- Error handling for all not-found cases can be made more consistent.
- The repository exposes a status finder, but the controller does not currently expose a status-filter endpoint.

13. FUTURE ENHANCEMENTS

1. Add student, staff, and administrator authentication.
2. Implement role-based authorization.
3. Add a web frontend with complaint forms and tracking screens.
4. Add an administrator dashboard for assignment and status management.
5. Add notifications through email or push messages.
6. Add status-history records instead of storing only the current status.
7. Add pagination, sorting, and date-range filters.
8. Store uploaded images in object storage and retain secure URLs.
9. Add audit logging for update and deletion operations.
10. Add comprehensive controller, service, and repository tests.
11. Restrict CORS origins for production use.
12. Move database secrets to environment variables.
13. Add API documentation using OpenAPI or Swagger.
14. Add analytics for complaint frequency and resolution time.
15. Add priority levels and assignment to responsible departments.

14. CONCLUSION

CampusCare provides a practical foundation for digital campus complaint management. The application replaces informal complaint recording with a centralized REST-based service that supports creation, retrieval, tracking, filtering, updating, and deletion. Its Spring Boot layered architecture and MySQL persistence make the implementation understandable, maintainable, and extensible.

The current project successfully demonstrates the core backend workflow. With the addition of authentication, frontend screens, notifications, stronger production security, and expanded testing, CampusCare can be developed into a complete campus service-management platform.

15. REFERENCES

1. Spring Boot Reference Documentation:
   https://docs.spring.io/spring-boot/
2. Spring Framework Documentation:
   https://docs.spring.io/spring-framework/reference/
3. Spring Data JPA Documentation:
   https://spring.io/projects/spring-data-jpa
4. Jakarta Bean Validation Specification:
   https://jakarta.ee/specifications/bean-validation/
5. MySQL Documentation:
   https://dev.mysql.com/doc/

6. Apache Maven Documentation:
   https://maven.apache.org/guides/

APPENDIX A: PROJECT SOURCE STRUCTURE

```
src/main/java/com/campuscare/campuscare/
    CampuscareApplication.java
    controller/ComplaintController.java
    exception/GlobalExceptionHandler.java
    model/ComplaintCategory.java
    model/complaint.java
    repository/ComplaintRepository.java
    service/ComplaintService.java

src/main/resources/
    application.properties

src/test/java/com/campuscare/campuscare/
    CampuscareApplicationTests.java
```

APPENDIX B: PROJECT FILE REFERENCES

The application entry point is implemented in CampuscareApplication.java.
Complaint API routes are implemented in ComplaintController.java.
Business operations are implemented in ComplaintService.java.
Database access is implemented in ComplaintRepository.java.
The complaint data model is implemented in complaint.java.
Validation error handling is implemented in GlobalExceptionHandler.java.
Database and application settings are stored in application.properties.